

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Yuvasri B	Reg. No.:	192421231
Section / Batch:	BTECH IT	Date:	22/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I Yuvasri B /192421231 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yuvasri B

Q40. Advantages and Disadvantages of DP vs Greedy

1. Problem Selected: 0/1 Knapsack Problem

The 0/1 Knapsack problem is an optimization problem where we have a set of items, each having a weight and a value. We must select items so that the total weight does not exceed the given capacity while maximizing the total value.

Consider the following example:

Item	Weight	Value
A	2 kg	₹20
B	3 kg	₹30
C	4 kg	₹50
D	5 kg	₹60

Knapsack capacity = 7 kg

The problem can be solved using both Dynamic Programming and a Greedy approach.

2. Dynamic Programming Approach

Dynamic Programming considers both possibilities for every item:

- Include the item.
- Exclude the item.

It stores the solutions of smaller subproblems and uses them to find the final optimal solution.

Steps

1. Create a table based on the number of items and available capacity.
2. Initialize the first row and column with 0.
3. Consider each item one by one.
4. If an item cannot fit, keep the previous value.
5. If it can fit, calculate:
 - Value without the item.
 - Value with the item.
6. Store the maximum of the two values.
7. The final table entry gives the maximum possible value. **Optimal Solution** For capacity 7 kg:
 - Item A + Item C = 2 + 4 = 6 kg, value = ₹70
 - Item B + Item C = 3 + 4 = 7 kg, value = **₹80**
 - Item B + Item D = 3 + 5 = 8 kg, not possible
 - Item A + Item D = 2 + 5 = 7 kg, value = ₹80 Therefore, the maximum value is:

Optimal Value = ₹80

3. Greedy Approach

A common greedy strategy is to select items according to their **value-to-weight ratio**.

Formula:

$$\text{Value/Weight} = \text{Value} \div \text{Weight}$$

Item	Weight	Value	Value/Weight
A	2	₹20	10
B	3	₹30	10
C	4	₹50	12.5
D	5	₹60	12

Greedy
order: **C**
→ **D** →
A/B

Select C:

Weight = 4 kg
Value = ₹50

Remaining capacity = 3 kg.

Select B:

Weight = 3 kg
Value =
₹30 Total:

Weight = 7 kg
Value = **₹80**

In this particular example, Greedy also gives the optimal value of **₹80**.

However, Greedy does **not guarantee optimality for every 0/1 Knapsack instance**. DP is preferred when an exact optimal solution is required.

4. Comparison of DP and Greedy

Feature	Dynamic Programming	Greedy
Strategy	Solves overlapping subproblems	Makes the best local choice
Optimality	Guarantees optimal solution for 0/1 Knapsack	May produce suboptimal solution
Time Complexity	$O(nW)$	$O(n \log n)$ due to sorting
Space Complexity	$O(nW)$	$O(n)$
Memory Usage	High	Low
Speed	Relatively slower	Faster
Implementation	More complex	Simple
Scalability	Limited when capacity is very large	Generally more scalable
Decision Making	Considers future consequences	Considers current best choice

Suitable Example	0/1 Knapsack	Fractional Knapsack
Main Advantage	Optimality	Efficiency
Main Disadvantage	High memory/computation	May be suboptimal

Here, n represents the number of items and W represents the knapsack capacity.

5. Advantages of Dynamic Programming

1. Provides an optimal solution for problems such as 0/1 Knapsack.
2. Handles overlapping subproblems efficiently.
3. Avoids repeated calculations.
4. Can solve complex optimization problems systematically.
5. The optimal solution can be reconstructed from the DP table.
6. **Disadvantages of Dynamic Programming**
 1. Requires high memory for storing the DP table.
 2. Can be computationally expensive.
 3. Implementation is more complicated than Greedy.
 4. Performance decreases when the state space becomes very large.
 5. May not scale well for very large capacities or numbers of states.

7. Advantages of Greedy

Algorithm 1. Simple and easy to implement.

2. Usually faster than DP.
3. Requires less memory.
4. Suitable for large input sizes when the greedy property is valid.
5. Makes decisions quickly without maintaining a large table.

8. Disadvantages of Greedy Algorithm

1. Does not always produce an optimal solution.
2. A wrong early decision cannot normally be reversed.
3. Requires the problem to satisfy the greedy-choice property.
4. Not suitable for all optimization problems.
5. For 0/1 Knapsack, it can give a suboptimal result.

9. Correctness Analysis Dynamic Programming

DP is correct for the 0/1 Knapsack problem because it considers both possibilities—selecting and not selecting each item—and stores the best solution for every smaller subproblem. Therefore, the final result is guaranteed to be optimal.

Greedy

Greedy is correct only when the problem has the **greedy-choice property**. For example, Greedy gives an optimal solution for the **Fractional Knapsack problem** because items can be divided.

For the **0/1 Knapsack problem**, items cannot be divided. Therefore, selecting the item with the highest value-to-weight ratio does not always guarantee the optimal solution.

10. Computational Complexity

Dynamic

Programming Time

Complexity:

$O(nW)$

Space Complexity:

$O(nW)$

DP requires more memory because it stores solutions for different combinations of items and capacities.

Greedy

Sorting the items based on their value-to-weight ratio requires:

$O(n \log n)$

Selecting the items takes:

$O(n)$

Therefore, overall:

Time Complexity = $O(n \log n)$ Space Complexity = $O(n)$

Thus, Greedy is generally faster and uses less memory.

11. Scalability

Dynamic Programming becomes difficult to scale when the number of items or the capacity becomes very large because the DP table grows with $n \times W$.

Greedy algorithms generally scale better because they require sorting and then making a sequence of simple decisions.

However, scalability should not be considered alone. If Greedy cannot guarantee an optimal solution, its speed may come at the cost of solution quality.

12. Insights for Selecting the Appropriate Paradigm

The choice between DP and Greedy depends on the problem requirements.

Choose Dynamic Programming when:

- An optimal solution is required.
- The problem has overlapping subproblems.
- The problem has optimal substructure.
- The state space is manageable.
- Memory and computation are available.

Choose Greedy when:

- A greedy-choice property can be proven.

- Fast execution is important.
- Memory usage should be low.
- The problem allows local optimal choices to lead to a global optimum.
- The input size is very large.

13. Conclusion

Dynamic Programming and Greedy algorithms provide different trade-offs between optimality and efficiency.

Dynamic Programming generally requires more memory and computation but can guarantee an optimal solution for problems such as 0/1 Knapsack.

Greedy algorithms are simpler, faster, and more memory-efficient, but they may produce suboptimal solutions when the greedy-choice property does not hold.

Therefore, DP should be selected when solution optimality is the priority, while Greedy should be selected when efficiency and scalability are more important and the greedy strategy is proven to be correct.

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated

Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand
---------	-----	--	-----------------------------------	---	---

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____